

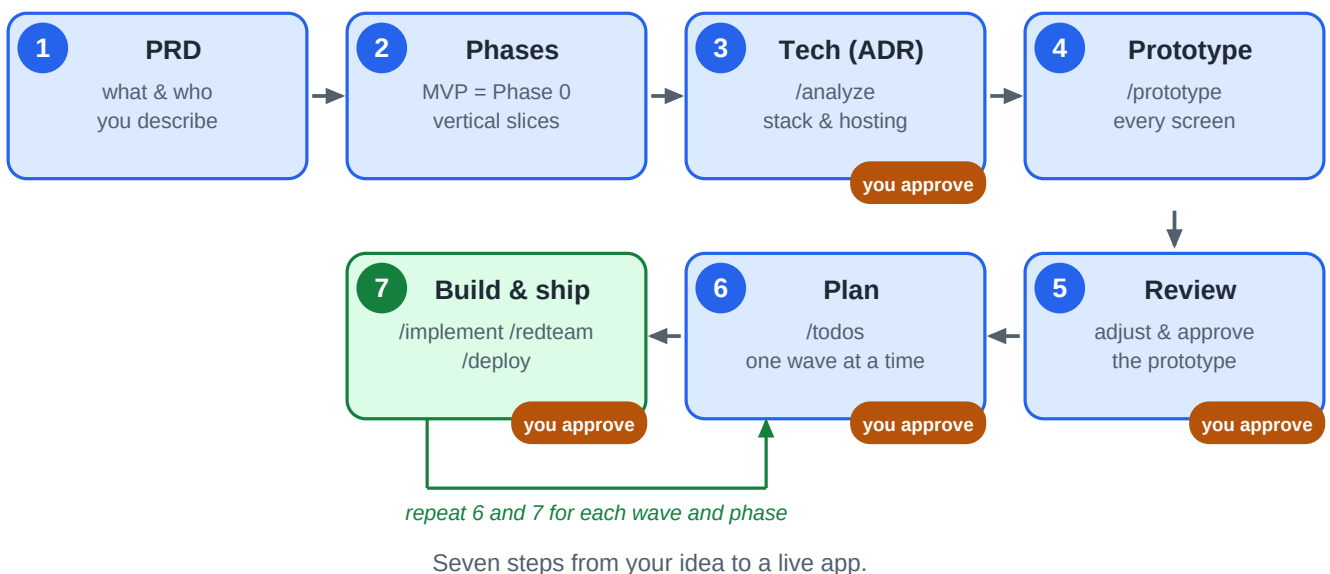
Build your first app with Claude Code

A step-by-step guide to code-starter-template for people new to vibe coding

You will not write code yourself. Your job is to **describe** what you want, **review** what Claude shows you, and **approve** it. Claude does the building, and the built-in rules in this template make sure the work is planned, tested and reviewed before it reaches your users.

The example builds a web app with **Next.js** for the screens, **Python** for the logic behind them, and **Postgres** for storing data, running in **Docker Desktop** on your computer. That covers most applications except phone apps sold through an app store.

The journey at a glance



You approve at a few fixed points: the technology and hosting choice (step 3), the prototype (step 5), each plan (step 6), and each preview and each time the app goes live (step 7). In between, Claude works on its own and reports what it did.

What it costs and how long it takes

- **Claude:** you need a paid Claude plan (Pro, Max, Team or Enterprise); the free plan does not include Claude Code. Steps 3 and 7 use a lot. On Pro, expect to pause at the usage limit in the middle of a step, especially in standard mode; Max gives more room. Wait for the reset, then type *Carry on where you stopped*.
- **GitHub:** free. Private projects get a monthly allowance of time for the automatic checks; if they stop running, ask *Have we used up the free check time?*
- **Docker Desktop:** free for personal use, education and small businesses; larger companies need a paid subscription, so check Docker's terms if you use it at work.
- **Hosting** (putting the app on the internet): often \$5 to \$30 a month, plus about \$10 to \$20 a year for your own web address. Claude gives the exact figure and a cheaper option, and waits for your OK before buying anything.
- **Time:** setting up your computer takes 30 to 60 minutes; each later step takes minutes to hours.

Before you start (once)

1. Create a free account at github.com and sign in.
2. Install the **Claude desktop app** from claude.com/download and sign in with your Claude account. (Prefer a terminal? See *Using a terminal instead* below.)
3. In the desktop app, open the **Code** tab, choose **Local**, and pick a folder for your projects, for example *Documents/Projects*. Then type: *Help me install Git, GitHub CLI, Node.js 22 or newer, Python and Docker Desktop on this computer. Go one at a time and tell me when each one works.* (See *What installing looks like* below.)
4. Type: *Log me in to GitHub, with permission to set up automatic checks.* Claude shows a short one-time code and opens a GitHub page (or gives you its address). Type the code on that page, then click **Authorize**. The permission matters: without it, GitHub refuses the automatic checks the first plan sets up.
5. Type: *Set my name for saved work to (your name), and use my private GitHub email address.* Git stamps this name on every snapshot; the private address keeps your real email out of the project's history.
6. On GitHub, open github.com/wailuen/code-starter-template and click **Use this template**. Give your new repository your project's name and choose **Private**, so only you (and people you invite) can see it.
7. Back in Claude, type: *Copy my GitHub repository <the link to your new repository> onto this computer.* Claude tells you the folder it made.
8. Start a **new session** in the Code tab and pick **that** folder. Always open this folder from now on.
9. Type `/start` for a short orientation, then `/doctor` to check your computer. Before step 3, `/doctor` says some checks are *not filled in yet* — that is normal; run it again after step 3.

What installing looks like

Installers are not fully automatic. Some open their own window or ask for your computer's password: type the password in that window, never in the chat. On a Mac, the first use of Git may open an Apple window offering to install developer tools; click **Install** and wait, it can take a while. Docker Desktop asks you to accept its terms and may ask you to restart the computer. If Claude asks you to paste something into the **Terminal** app, it tells you exactly what to paste. If an install will not work through Claude, use the official download pages: git-scm.com/downloads, cli.github.com, nodejs.org, python.org/downloads and docker.com/products/docker-desktop, then tell Claude it is done.

Good to know

`/start` describes the work in six broad stages; this guide's seven steps fit inside them. Follow this guide. This project has its own `/doctor`, which checks your computer for this project, and its own `/design`, which you do not need to type at all. You never need to type a folder or file name either: Claude knows where each file belongs.

Using a terminal instead

A *terminal* is a window where you type commands (on a Mac, the **Terminal** app). Install Claude Code with the steps at code.claude.com/docs/en/quickstart, then type `cd` followed by your project folder, press Enter, and type `claude`. Everything else in this guide is the same.

Talking to Claude

- Type in the box at the bottom and press **Enter** to send. For a new line without sending, press **Shift+Enter**.
- A command starts with `/` as the very first character, for example `/analyze`. Words after it are passed to the command. Type `/` on its own to see the list.
- To stop Claude in the middle of something, press **Esc**. To finish a session, type `/exit` or close the window.

When Claude asks for permission

This template does not pre-approve any commands, so Claude may ask for permission often, especially at first. Most requests are normal building work — installing parts, running tests, saving your work to GitHub — so allow them. If it offers to stop asking for a kind of command, that is fine for routine ones such as `git`, `gh`, `node` or the tests. Say no and ask *Why do you need to do that?* if it wants to delete files outside your project, spend money, change your GitHub account settings or put the app live. If you cannot tell, ask *Explain that in one sentence*. Saying no stops that one step and Claude waits for you.

Words you will see

Word	Meaning
PRD	Product requirements document: what the app does, for whom, and why
ADR	Architecture decision record: a short note of a technical choice and the reason for it
MVP	Minimum viable product: the smallest version people can actually use
Vertical slice	A small feature that works end to end: screen, logic and data together
Responsive design	One layout that rearranges itself to fit a phone, tablet or computer screen
Mobile-first	Design the phone layout first, then widen it for bigger screens
Prototype	Clickable pages that look like the finished app but have nothing behind them, for trying the design before it is built
Repository	Your project's folder on GitHub
Clone / copy	Make a copy of the repository on your computer
Commit	Save a snapshot of the project, with a short note of what changed
Push	Copy your snapshots to GitHub
Branch	A separate copy where unfinished work happens, so the main version stays safe
Merge	Join reviewed work into the main version
Pull request	GitHub's page for proposing a merge; Claude opens it for you
Wave	One batch of planned work; a phase is built in one or more waves
Todo	One piece of work inside a wave, with a clear list of what “done” means
Deploy	Put the app on the internet for your users

Step 1 — Write your PRD

Talk the idea through with Claude until it is clear. A paragraph is enough to start.

Type this

I would like to create a PRD for the below:

(Describe your idea in your own words: who will use it, what problem it solves, the main things a user should be able to do, and anything it must never do. Example: a private site where my family can share recipes with photos and find them again easily.)

Claude asks questions. Answer them, and ask Claude to explain anything you are unsure about. When the PRD reads right to you:

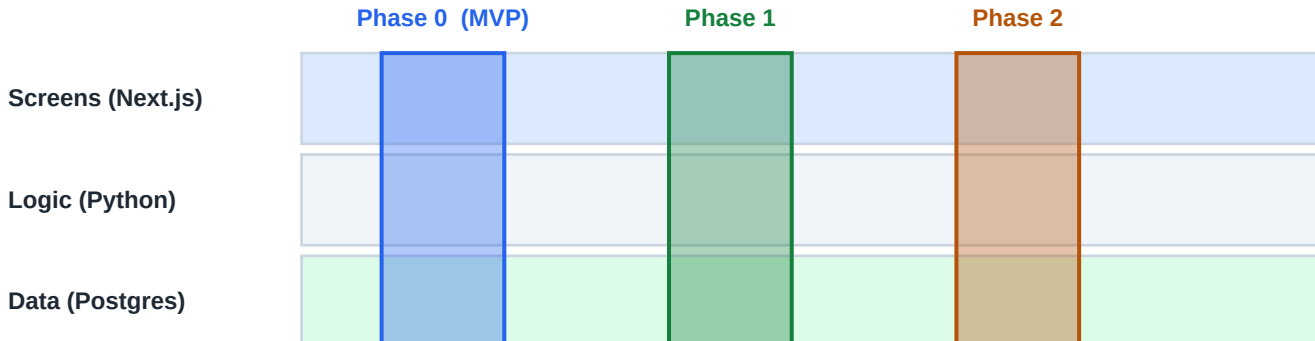
Type this (use your own project name)

Save this PRD as the project brief for my project called *Family Recipes*.

Step 2 — Split the PRD into phases

Type this

Structure the PRD by phase, with the MVP as Phase 0, then Phase 1, Phase 2 and so on. Each phase must be a vertical slice: something a user can see and use, with the screens, backend and database working together. Keep Phase 0 small. Update the project brief.



Each phase cuts through every layer, so every phase ends with something you can click through and try.

Why

A good Phase 0 has one to three things a user can do from start to finish. For a family recipe site: family members sign in, add a recipe with a photo, and see everyone's recipes; comments, search and printing wait for Phase 1. If Phase 0 has more than three things, ask *Make Phase 0 smaller*.

Step 3 — Decide the technology (ADR)

`/analyze` turns your brief into a proper plan: it researches, writes the specifications and records the technical decisions.

Type this

`/analyze` Let's build the ADR. I would like this application to be based on a Next.js frontend, a Python backend and Postgres as the database. Everything runs locally in development, with Postgres in Docker Desktop. Use the project brief and its phases.

This step can take from 30 minutes to a couple of hours. Claude will:

- research similar apps — for a private family or hobby app, add *Keep the market research short* to the prompt;
- write the specifications and the decision records (the ADR), and fill in the project's commands so later steps never guess;
- recommend where the app will run when it goes live, with its monthly cost;
- ask you to choose **light** or **standard** mode. **Light:** only you will use it, nothing important is stored, no money moves — fewer reviews. **Standard:** other people will sign in with their email, or you store real personal data or money — full reviews. If unsure, choose standard.

It is finished when Claude says the analysis is done and asks you to approve. Claude may upload the plan to GitHub as a *pull request* — that is normal.

You decide

Approve the technology, where the app will run and what it costs each month, and the mode. If you do not know, say *Recommend one, explain the trade-off in plain words, and I'll go with your recommendation*.

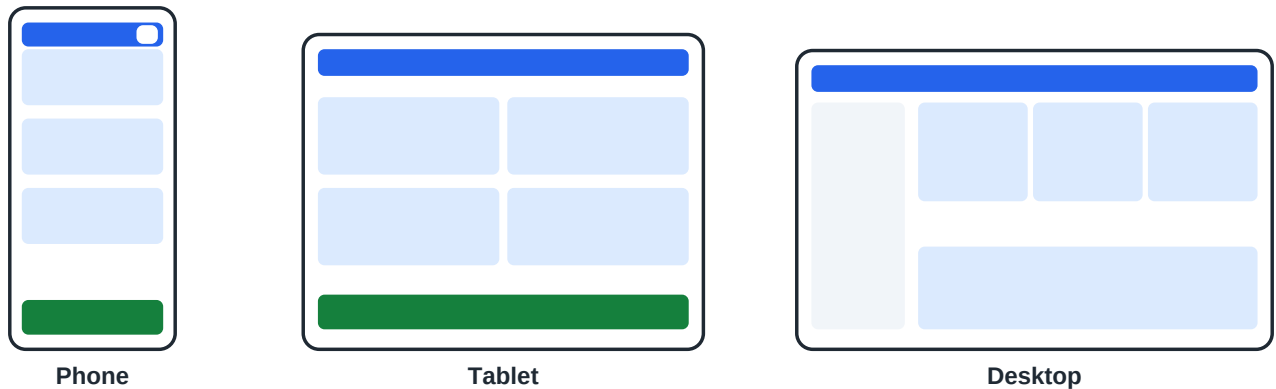
Now run `/doctor` again: it can check Python and Docker Desktop this time.

Step 4 — Design every screen (responsive, mobile-first)

Type this

/prototype Design every screen of every phase in the PRD.

The prototype covers your **whole PRD — every phase** — so you see the complete app before any of it is built. Claude first asks about the look you want: the feel, colours, apps you like, the device your users mostly hold. Answer in your own words, or say *you choose*. Then it lists every screen and builds them as clickable pages, designed for a phone first and rearranging themselves for tablets and computers.



Responsive design: one design that fits every screen size. Mobile-first: the phone layout is designed first.

Before showing you, Claude checks every screen at five widths in a browser it can control. It may ask to install one first (a large download). If it says the screen check is *owed*, type *Install what you need and run the screen check*. Then it tells you how to open the prototype — or ask *Open the prototype in my browser for me*. The start page lists every screen by phase. Each screen has a **phone · tablet · desktop** link that shows all three sizes side by side. To try it on your real phone, ask *How can I open this on my phone?* If your computer then asks whether to allow incoming connections, allow it: that is how your phone reaches the preview.

Good to know

Claude may use Claude Design (on the Pro, Max, Team and Enterprise plans; on Enterprise, your admin must turn it on). If it says it cannot reach Claude Design, type `/design - login` and finish the sign-in in your browser — or simply carry on: Claude builds the same clickable pages either way, and those pages are what gets approved.

Step 5 — Review the prototype, then approve it

Walk through every screen as if you were a new user. Be specific about what you want changed. Type these two for every app:

For every app

Show me every screen when there is no data yet, while it is loading, and when something goes wrong. Write the messages in plain, friendly words.

For every app

Walk me through the app as a brand-new user, from opening it to finishing the main task. Point out any step where I would get stuck or confused, then fix those steps.

These three are examples — rewrite them about your own screens:

Example

On the home screen the most important information is hard to find. Make it the first thing I see on a phone, and move the filters below it.

Example

Sign-up asks for too much. Keep only name, email and password, and ask for the rest after the first login.

Example

On a phone, the menu covers half the screen. Make it a simple menu button that opens a full-screen list, and keep the main buttons within thumb reach at the bottom.

Repeat until you are happy. Changes are cheap now and expensive after the code is built. If a change adds or drops a feature, Claude says so and updates the PRD with your OK.

Approve it

When you say you are happy, Claude asks you to approve, explaining what yes and no mean. Answer in your own words, for example:

Type this

Yes, I approve every screen for all phases.

You decide

Approve the prototype. Claude asks your name the first time and records it with the date and your words, and from then on plans and builds the screens to match it; anything that must differ is asked first. You can approve some phases now and the rest later. To change an approved design later, type `/prototype` again.

Step 6 — Plan the work

Type this

/todos Plan Phase 0 from the PRD phasing and the approved prototype. Each feature todo must deliver a working vertical slice — screen, backend and database together.

Claude ties each piece of work to the approved screens it builds, and checks the result against them at phone, tablet and desktop size. A phase may be split into a few batches (waves). Claude plans one wave at a time, shows it to you, and asks four questions:

- Does it cover everything you described?
- Is anything there that you did not ask for?
- Is anything missing that you expected?
- Does the order make sense?

If the plan is hard to read, ask *Explain each item as what I will be able to do when it is finished*. If you cannot judge the order, ask *Why this order?*

The first plan also asks to switch on automatic checks on GitHub — say yes. It may mention locking the main version with a paid GitHub plan; you can say no and everything still works. Before a wave starts, Claude may also ask you to get a key from another service (for example a sign-in or email service); it tells you each click and where to put the key safely.

You decide

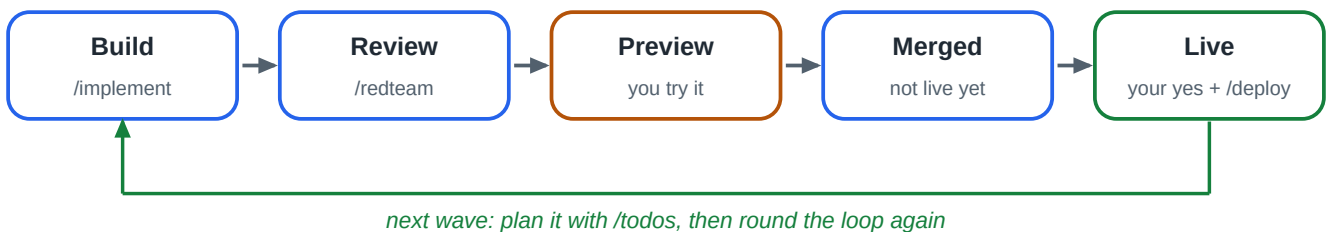
Approve the plan. Approving freezes this wave's work. You can still change your mind: say so, and Claude explains what changes and asks you to approve a new plan.

Step 7 — Build, review and go live

First open **Docker Desktop** and wait until it says it is running (on a Mac, a whale icon at the top of the screen). Do this at the start of every working session: the tests need it. Then:

Type this

`/implement` Start with the first todo of the approved plan.



Every wave goes round this loop. Nothing reaches your users until you type `/deploy`.

1. **Build** — each todo is built test-first; in standard mode an independent reviewer also checks each todo. Claude says when a todo is done and what changed. Each `/implement` builds one todo; type it again, or say *Carry on with the next todo*.
2. **Review** — when every todo of the wave is built, Claude runs `/redteam` (or type it yourself). It reviews everything together, security included. This can take a while.
3. **Preview** — Claude shows you what changed and how to try it, and asks if it matches what you wanted. Answer yes, or say what is wrong. Claude joins the work into the main version either way, and that does not put it live. Your yes is what lets `/deploy` put this wave live later; after a *no*, `/deploy` refuses until Claude has fixed what you said or you have agreed a change of plan.
4. **Go live** — the first time, type `/deploy --onboard`. Claude confirms the hosting you chose in step 3, tells you what it costs, walks you through creating the hosting account and adding a payment card, and sets up an alert that reaches you if the site goes down. It may ask you to install the hosting company's tool and log in to it, the same way you logged in to GitHub, and to paste secret keys into the hosting company's website (never into the chat). You will need a card, an email or phone for alerts, and about an hour. After that, type `/deploy` to put the new version live. Only you start a deploy.

Good to know

While Claude builds and reviews, browser windows may open and click by themselves. That is Claude testing the app. Leave them alone until Claude says it is done.

Try it on your computer

Before anything goes live, ask *Start the app on my computer so I can try it*. Claude gives you an address such as `http://localhost:3000` — open it in your browser. Only you can see it; it is not on the internet. When you are done, say *Stop the app*.

Important

Do not connect your project to a hosting website on your own, even if the site offers a *Connect GitHub* button. Let `/deploy` set it up, so unreviewed work never goes live. During `/deploy --onboard`, Claude may ask you to click such a button; do it then, as Claude walks you through it step by step.

When a wave is finished, type `/ws`. It tells you whether the next step is `/todos` (plan the next wave of this phase, or the next phase) or `/deploy`.

Saving your work (commit and push)

Claude commits as it works, on a side copy (a branch), and opens a pull request when the work is ready to join the main version. Still, ask for it whenever you finish something or stop for the day (if a review is running, let it finish first):

Type this

Commit and push my work.

Why it matters

A safety net: if something goes wrong later, Claude can bring back an earlier snapshot. **A backup:** your work is safe on GitHub even if your computer is lost. **Sharing:** anyone you invite can see the latest version of the project. Pushing does not change your live app: nothing reaches your users until you type `/deploy`.

Tips for beginners

- **Just tell Claude what you want, in your own words.** You do not need technical terms; Claude asks when something is unclear.
- **Do not be afraid to make mistakes.** Code changes can be undone — say *That's not what I meant, undo it* and try again. Claude asks you first before anything that cannot be undone, such as deleting data, spending money or putting the app live.
- **Be specific.** “Make the button bigger and green, at the bottom of the screen” beats “make it nicer”.
- **Ask “why?”** Claude must explain its choices in plain words. If you do not understand, say so.
- **Try it yourself** at every preview, on your phone as well as your computer.
- **Keep phases small.** A small working app beats a big unfinished one.
- **Never paste passwords or secret keys into the chat.** Claude tells you where they go safely.

When you are stuck

Situation	What to do
Something is broken on the live site	Type <code>/fix</code> and say what went wrong. If the site is down or data is at risk, it first asks whether to undo the last update
Claude shows an error you do not understand	<i>What does this mean for me, and what should I do?</i>
Claude asks a question you cannot answer	<i>I don't know — recommend one and explain why.</i>
You reached your usage limit	Wait for it to reset, then <i>Carry on where you stopped.</i>
Where am I? What is waiting for me?	<code>/ws</code>
You want to change direction	Say so in plain words; Claude explains the impact and asks before changing the plan
Claude asks to change its own working rules	After each wave Claude improves its own rules from what went wrong, and some changes wait for your OK. Ask <i>What changes for me if I say yes?</i> If it only makes Claude more careful, say yes; if you are unsure, say <i>not now</i>
Ending a working session	<i>Commit and push my work</i> , then <code>/wrapup</code> (it saves notes on this computer for your next session, in that order so it can also save lessons). It can take a few minutes and may open a pull request on GitHub
Starting a new session	Open Docker Desktop, open the project folder, and type <i>Where did we leave off?</i> (or <code>/ws</code>). Claude waits for your first message before it reads its notes. To reopen an earlier conversation, pick it from the list in the desktop app (in a terminal: <code>c laude --continue</code> inside the project folder)